



THE UNIVERSITY OF AUCKLAND
NEW ZEALAND

COMPSCI 702FC

Software Measurement

What do you want to measure? Goal-based Framework

Department of Computer Science, UoA

Dr. Emilia Mendes (emilia@cs) (supervisor)

Dr. Ewan Tempero (e.tempero@cs)

<http://www.cs.auckland.ac.nz/compsci702s1c/>

Goal-based Measurement

- ◆ The primary question in goal-based measurement is:
 - “What do we want to know or learn?”
instead of “what measures shall we use?”
- ◆ The answers will depend on the organisation’s goals, therefore no fixed set of measures is universally adequate.

Goal-based Framework

- ◆ The framework is based on three principles:
 - Classifying the **entities** to be examined
 - Determining relevant measurement **goals**
 - Identifying the **level of maturity** an organisation has reached

What Entities to measure?

◆ Three classes of entities:

- **Processes** – collections of software related activities usually associated with some timescale.
 - ◆ Different SW processes: development, maintenance, testing, reuse, configuration and management process etc.
- **Products** – any artefacts, deliverables or documents that result from a process activity.
- **Resources** – entities required by a process activity.

Attribute Types

- ◆ Within each entity class, there can be two types of attributes:
 - **Internal attributes** of a product, process or resource are those that can be measured by examining the product, process or resource on its own, separate from its behaviour.
 - **External attributes** of a product, process or resource relates to its environment. The behaviour of the product, process or resource is important, rather than the entity itself.

Sample Process Measures (1)

Process Entities	Attributes	Possible Measures
Development process	Elapsed time	Number of working days
	Milestones	Calendar dates
	Effort	Staff hours
	Phase containment	Percentage of total defects found in same phase where it was introduced
	Process compliance	Percentage of tasks complying with standard procedures or directives
	Performance	Number of tests passed divided by number of tests executed
Test process	Volume	Number of tests scheduled
	Progress	Number of tests executed Number of tests passed

Sample Process Measures (2)

Process Entities	Attributes	Possible Measures
Detailed design	Elapsed time	Number of working days
Maintenance	Cost	NZ Dollars per year Staff hours per change request
Change request backlog	size	Number of change requests awaiting service Estimated effort (person hours) for pending requests

Sample Product measures (1)

Product Entities	Attributes	Possible Measures
System	Size	Number of modules Number of bubbles in a data-flow diagram Number of use cases in a use case diagram Number of function points Number of physical source lines of code Number of memory bytes or words required (or allocated)
	Defect density	Defects per KLOC Defects per function point
Module	Length	Physical source lines of code Logical source statements
	Percent reused	Ratio of unchanged physical lines to total physical lines, comments and blanks excluded

Sample Product measures (2)

Product Entities	Attributes	Possible Measures
Unit	Number of linearly independent flowpaths	McCabe's complexity
Document	length	Number of pages
Lines of code	Statement type	Type names
	How produced	Name of production method
	Programming language	Language name
Defect	Type	Type names
	Origin	Name of activity where introduced
	Severity	An ordered set of severity classes
	Effort to fix	Staff hours

Sample Resource measures

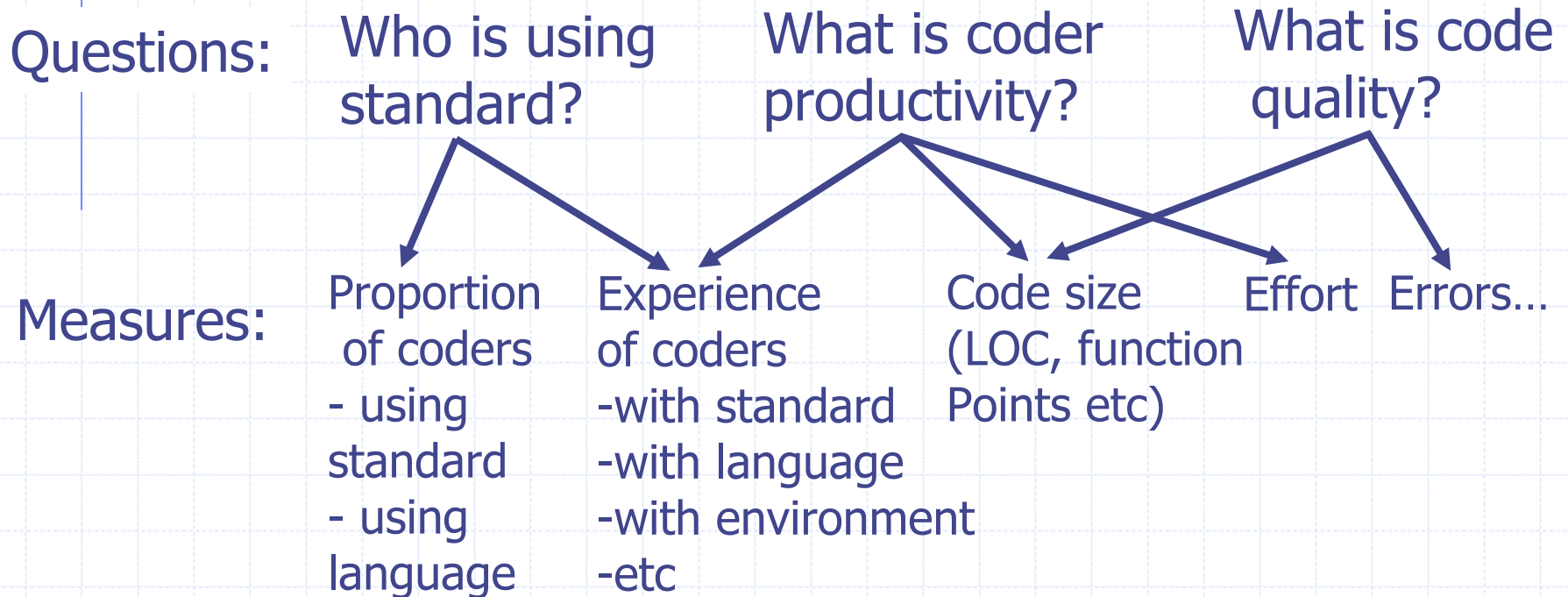
Resource Entities	Attributes	Possible Measures
Assigned staff	Team size	Number of people assigned
	Experience	Year of domain experience Year of programming experience
CASE tools	Type	Name of type
	Is used?	Yes/no (a binary classification)

GQM Approach (1)

- ◆ The Goal-Question-Metric (GQM) approach provides a framework involving three steps:
 - **Goal:** List the major goals of the development or maintenance project
 - **Question:** Derive from each goal the questions that must be answered to determine if the goals are being met
 - **Metric:** Decide what must be measured in order to be able to answer the questions adequately

GQM Example

Goal: Evaluate effectiveness of coding standard



Template for Goal Definition (1)

◆ **Purpose:** To (characterize, evaluate, predict, motivate etc) the (process, product, model, measure etc) in order to (understand, assess, manage, engineer, learn, improve etc) it.

- Example: To evaluate maintenance process in order to improve it.

Template for Goal Definition (2)

◆ **Perspective:** Examine the (cost, effectiveness, correctness, defects, changes, product measures etc) from the viewpoint of the (developer, manager, customer etc).

- Example: Examine the cost from the viewpoint of the manager.

Template for Goal Definition (3)

◆ **Environment:** The environment consists of the following: process factors, people factors, problem factors, methods, tools, constraints etc.

- Example: The maintenance staff are poorly motivated programmers who have limited access to tools.

Examples (1)

◆ Purpose:

- Determine a way to improve the productivity of our development team by evaluating their current productivity

◆ Perspective:

- Examine the team personality factors, expertise of the development organisation, team's analysis and design techniques, knowledge of programming languages used by our development team

◆ Environment and constraints:

- Payroll applications programming in C++
- 100 software developers with 5 or more years experience in C++
- Customers are businesses
- Do not maintain a reusable module database
- Examine new projects completed and sold from 1/1/2002 to 31/12/2004.

Examples (2)

◆ Purpose:

- Evaluate working environment in order to identify opportunities to improve the productivity of our development team

◆ Perspective:

- Examine the ratio of work time to break time of our employees, the accommodations, incentives, extra-curricular activities offered, workspace (room and desk size, ventilation) where our employees work from the point of view of the employees themselves

◆ Environment and constraints:

- Payroll applications programming in C++
- 100 software developers with 5 or more years experience in C++
- Customers are businesses
- Do not maintain a reusable module database
- Examine new projects completed and sold from 1/1/2002 to 31/12/2004.

Examples (3)

◆ Purpose:

- Evaluate the impact of various CASE tools on the productivity of the development team

◆ Perspective:

- Examine the effectiveness of using various CASE tools to help in the development of our product from the point of view of the developers and testers

◆ Environment and constraints:

- Payroll applications programming in C++
- 100 software developers with 5 or more years experience in C++
- Customers are businesses
- Do not maintain a reusable module database
- Examine new projects completed and sold from 1/1/2002 to 31/12/2004.

GQM, Entities and Attributes

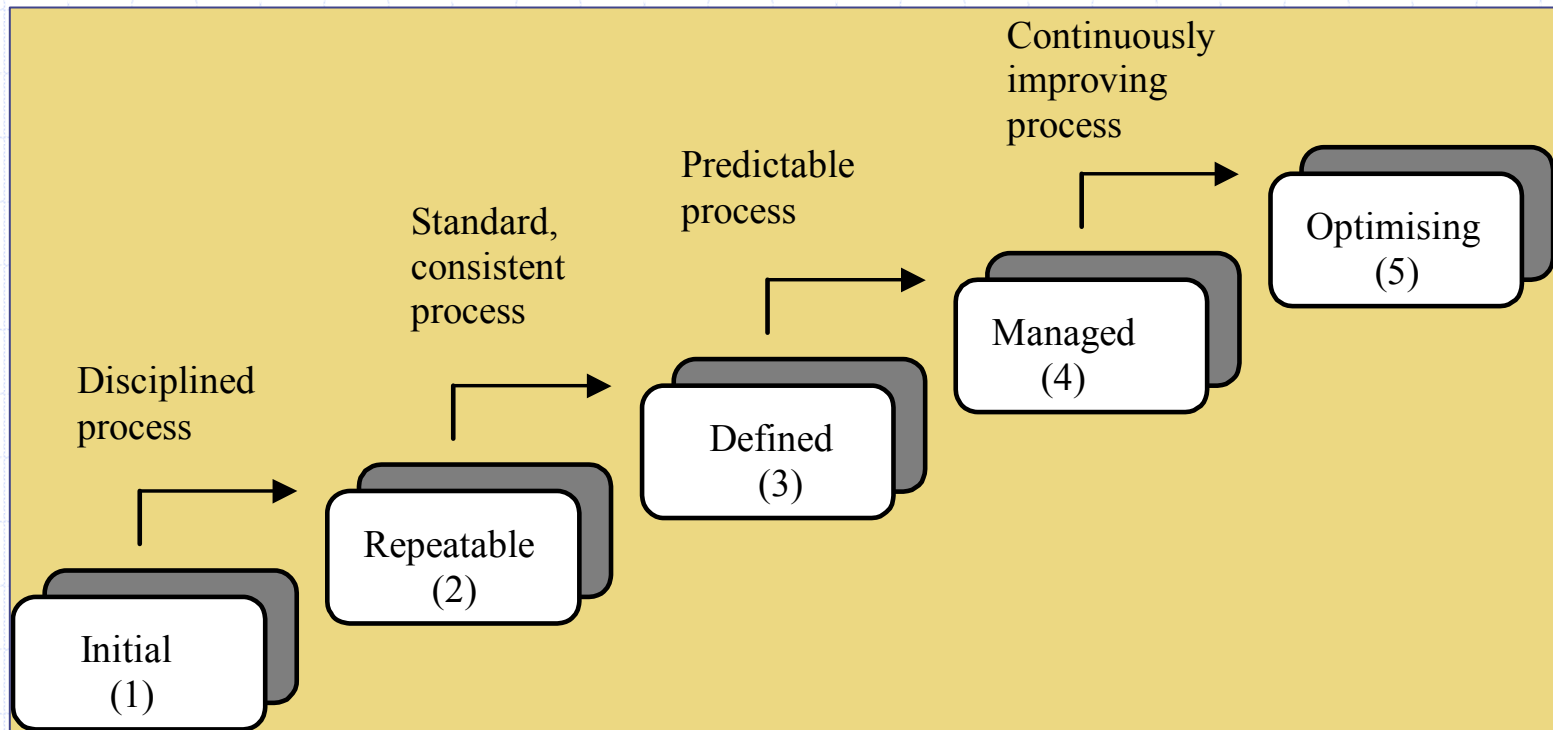
- ◆ We can relate the GQM template to the Entity&Attribute framework.
- ◆ A Goal or question can be associated with at least one pair of Entities and Attributes.
- ◆ Once a measure is obtained, it can be related back to questions and goals.

SEI CMM (Capability Maturity Model)

◆ Five levels of process maturity:

- Initial
 - ◆ Essentially uncontrolled
- Repeatable
 - ◆ Product management procedures defined and used
- Defined
 - ◆ Process management procedures and strategies defined and used
- Managed
 - ◆ Quality management strategies defined and used
- Optimizing
 - ◆ Process improvement strategies defined and used

CMM Levels



CMM Level 1

◆ Level 1 - The Initial Level

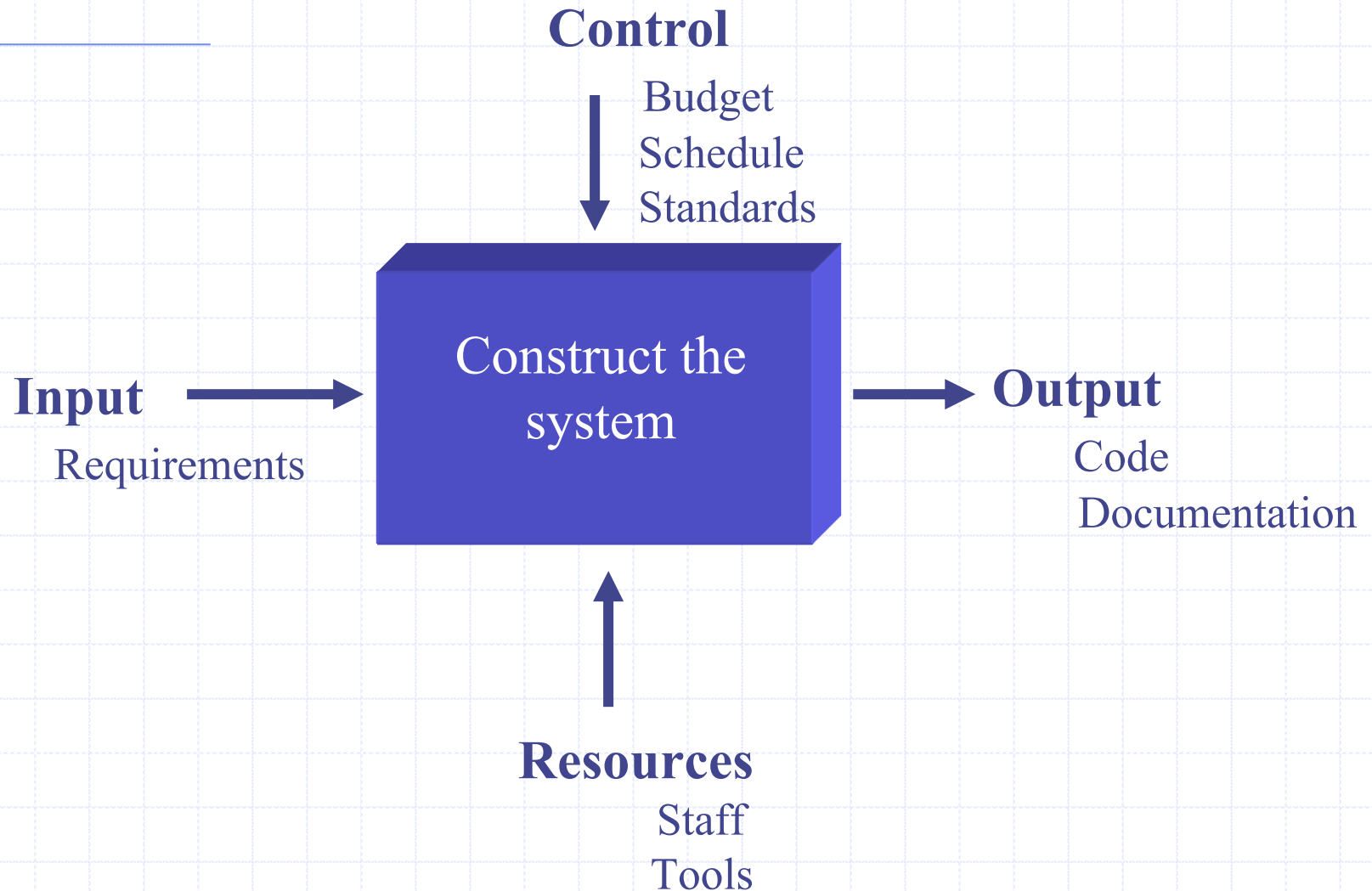
At the initial level, the organisation typically does not provide a stable environment for developing and maintaining software. Success depends entirely on having a very good manager and an experienced and effective software team. Inputs to the process are ill-defined; transition from inputs to outputs is undefined and uncontrolled. Similar projects may vary widely in their productivity and quality characteristics due to lack of adequate structure and control. Baselines are needed to provide a starting point for measuring improvement as maturity increases.

CMM Level 2

◆ Level 2 - The Repeatable Level

At the repeatable level the organisation establishes policies for managing a software project and procedures to implement those policies. New projects are planned and managed based on experience with similar projects and basic software management controls have been installed. Software process capability of level 2 organisations is disciplined because planning and tracking of the software project is stable and earlier successes can be repeated.

A repeatable Process (level 2)

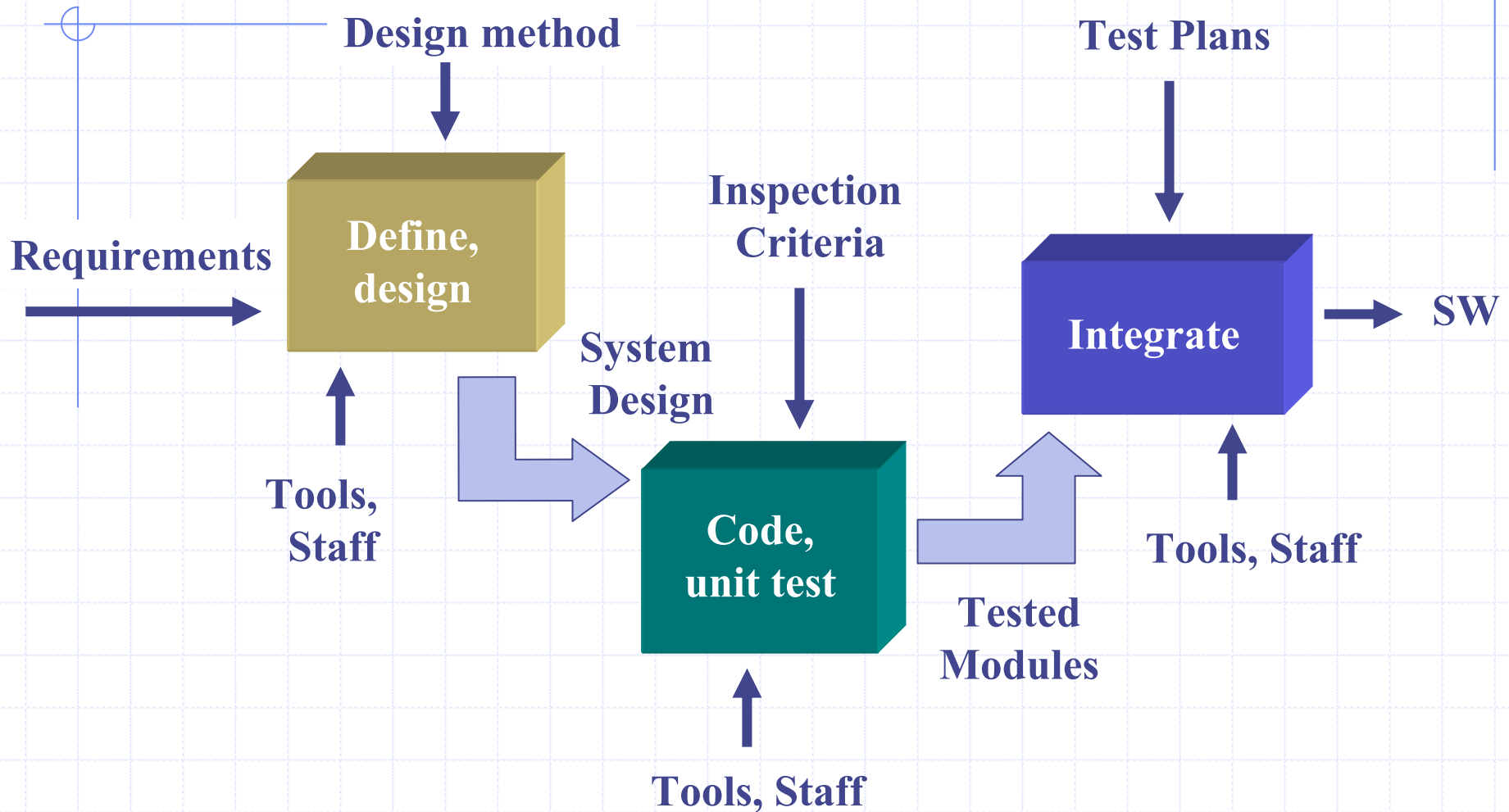


CMM Level 3

◆ Level 3 - The Defined Level

At the defined level the standard process for developing and maintaining software across the organisation is documented, including both software engineering and management processes, and these processes are integrated into a coherent whole. Management has good insight into technical progress on all projects because the software process is well defined. The software process capability of level 3 organisations is standard and consistent because both software engineering and management activities are stable and repeatable.

The Defined Level (level 3)

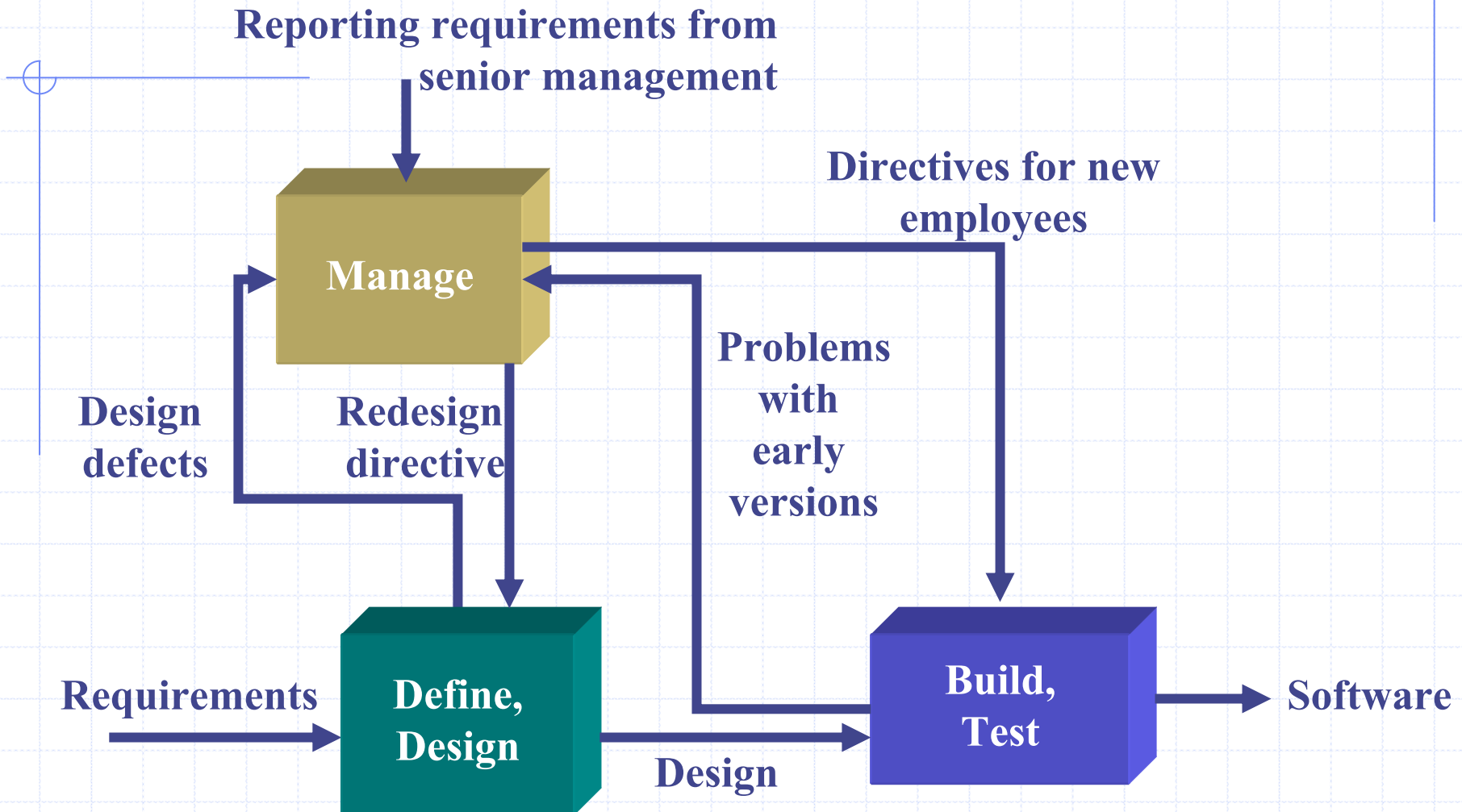


CMM Level 4

◆ Level 4 - The Managed Level

At the managed level, the organisation sets quantitative quality goals for both software products and processes. There is an organisational measurement program where productivity and quality are measured for important software process activities across all projects. The software process capability of level 4 organisations is quantifiable and predictable because the process is measured and operates within measurable limits.

The Managed Level (level 4)



CMM Level 5

❖ Level 5 - The Optimising Level

At the optimising level, the entire organisation is focused on continuous process improvement. Software project teams in level 5 organisations analyse defects in order to determine their causes. The software process capability of level 5 organisations improves continuously, striving to improve the range of their process capability. The spiral model is an example of such a dynamically changing process, responding from feedback from early activities in order to reduce risks in later ones.

Combining GQM with Process Maturity

- ◆ Since process maturity suggests that one can only measure what is visible, the incorporation of process maturity with GQM helps identify what measures will be most useful to the organisation at that particular moment in time.